

Accident Support Desk Complete Handoff Guide

This is the centralized handoff guide for cloning or transferring the Accident Support Desk SMS bot into another GoHighLevel account that is running the same personal-injury intake model.

The smaller Markdown files in this folder remain the editable source references. This file and the PDF are the one-file handoff version.

Master Links

- GitHub repo: <https://github.com/aileadzsudo/asdleads-sms-bot>
- Current production URL: <https://asdleads-sms-bot.onrender.com/>
- Secure database export link: SECURE_DB_EXPORT_LINK_TO_BE_FILLED_BY_OWNER
- Local handoff folder: docs/handoff/
- Sanitized env template: docs/handoff/HANDOFF_ENV_TEMPLATE.env

Critical Security Rule

Do not put live API keys, Slack tokens, GoHighLevel tokens, database URLs, Render secrets, or full customer/contact exports inside GitHub or this PDF.

The PDF explains how to transfer the system. The live contacts/messages database should be transferred separately as an encrypted Postgres dump through a secure link or password manager.

What To Hand To Another Operator

1. Access to the private GitHub repo or a private fork.
2. This PDF and the docs/handoff folder.
3. The sanitized env template.
4. A secure encrypted database dump link, if cloning historical data is required.
5. Their own GHL token, Slack token, OpenAI key, hosting account, Postgres database, calendar ID, and webhook secret.
6. The validation test plan, completed before DRY_RUN is turned off.

Update Strategy Summary

Best option: give the recipient a private fork of this repo and keep your repo as upstream. They can pull new tagged releases from you without sharing your production secrets or database.

Do not give a recipient automatic live deploys from your main branch unless they are part of your same internal company and you trust every change to hit their production immediately.

Table Of Contents

- [Quick Start](#)
- [Implementation Manifest](#)
- [Owner Export Checklist](#)
- [Recipient Setup Checklist](#)
- [Environment Variables](#)
- [GoHighLevel Workflows And Webhooks](#)
- [Database Transfer](#)
- [Validation Test Plan](#)
- [Repo Update Strategy](#)
- [Export Package Checklist](#)

Quick Start

Source file: docs/handoff/QUICK_START_FOR_NEW_OPERATOR.md

Quick Start For A New Operator

This is the shortest possible version.

What You Need

- A copy/fork of this repo.
- Render account.
- Render Postgres database.
- GHL location ID.
- GHL private integration token.
- GHL calendar ID.
- Slack bot token and channel IDs.
- OpenAI API key.

Steps

1. Deploy repo on Render.
 2. Add Postgres and `DATABASE_URL`.
 3. Add every env var from `HANDOFF_ENV_TEMPLATE.env`.
 4. Keep `DRY_RUN=true`.
 5. Open `/health`.
 6. Open `/integrations`.
 7. Create GHL ping webhook and confirm `/webhooks/ghl/ping-status`.
 8. Create all GHL workflows from `GHL_WORKFLOWS_AND_WEBHOOKS.md`.
 9. Test one fake lead through:
 - NR enrollment
 - inbound reply
 - booking
 - reminder
 - no-show
 - human outbound SMS
 - STOP
-
1. Set `DRY_RUN=false`.
 2. Test one real contact owned by your team.
 3. Start small batch only.

If Something Looks Wrong

1. Set `DRY_RUN=true`.
2. Pause GHL workflows.
3. Check `/health`.
4. Check `/integrations`.
5. Check dashboard issue page.
6. Check Render logs.
7. Run `npm test` locally before changing code.

Implementation Manifest

Source file: docs/handoff/IMPLEMENTATION_MANIFEST.md

Implementation Manifest

This file explains what has been built so another developer can understand the system quickly.

Core Runtime

- Runtime: Node.js 20+.
- Server entry: `src/server.js`.
- Start command: `npm start`.
- Build command: `npm run build`.
- Tests: `npm test`.
- Production storage: Postgres through `src/postgresStore.js`.
- Local/dev storage: JSON store through `src/store.js` when `DATABASE_URL` is not set.

Data Model

Production Postgres creates these tables automatically:

- `contacts` : full bot memory per GHL contact.
- `jobs` : scheduled outbound messages, reminders, re-engagement jobs, no-show jobs, watchdogs.
- `messages` : inbound, outbound, and human outbound messages.
- `escalations` : human escalation records and notification events.
- `decision_logs` : explanation of why the bot sent, skipped, paused, escalated, booked, or repaired.
- `webhook_events` : idempotency/dedupe for webhook events.
- `settings` : operational logs such as webhook diagnostics and integration checks.

Most detailed state lives inside JSONB `data` columns so new fields can be added without manual migrations.

Contact State Stored

Important contact fields include:

- `id`, `ghlContactId`, `name`, `phone`
- `timezone`, `timezoneSource`
- `leadSource`
- `engagementStatus`
- `qualificationProgress`
- `accidentDate`
- `faultAnswer`
- `medicalTreatmentAnswer`
- `preferredCallTime`, `preferredCallTimeIso`
- `backupCallTime`, `backupCallTimeIso`
- `appointmentId`, `appointmentType`, `appointmentTitle`
- `lastInboundMessage`, `lastOutboundMessage`
- `lastResponseTimestamp`, `lastOutboundTimestamp`
- `optOutStatus`
- `humanEscalationStatus`, `humanEscalationStage`
- `automationPaused`, `automationPauseReason`
- `currentSequenceName`, `currentSequenceDay`, `currentSequenceSlot`
- `noShowCount`, `repeatNoShow`, `previousAppointmentMissed`
- `lastLlmGateDecision`, `lastLlmGateReason`, and related LLM fields

Main Bot Flows

NR / No-Response Enrollment

Trigger:

- GHL no-response disposition.
- GHL NR tag.

Behavior:

- Checks terminal/pause tags.
- Sends Day 1 initial cold SMS immediately for fresh NR.
- Queues same-day aggressive follow-up and long-term cold outreach.
- Does not resend the initial cold SMS if the lead is already enrolled.

Cold Outreach

Used when the lead has not replied yet.

- Day 1 fresh leads can receive aggressive same-day messages if legally allowed.
- Long-term cold cadence continues up to 21 days.
- PM slot is 6 PM local time.
- Stop conditions: reply, opt-out, signed, NQ, manual hold, DND/SMS block, appointment, human escalation.

Inbound SMS Router

Trigger:

- `/webhooks/ghl/inbound-sms`

Behavior:

- Buffers quick consecutive messages.
- Checks STOP/opt-out first.
- Checks signed/NQ/manual hold.
- Detects lead-requested pause messages like "not a good time, I'll text when free, don't blow up my phone."
- Resumes from qualification memory instead of restarting.
- Uses deterministic parsers first.
- Uses LLM fallback/gate only where needed.

Lead-Requested Pause

Messages like "not a good time" plus "I'll text when I'm free" or "don't blow up my phone" do this:

- Send one polite acknowledgement.
- Cancel pending bot jobs.
- Create a human-visible escalation/Slack alert.
- Do not mark opt-out.
- Resume only when the lead texts back again.

Qualification Memory

Current simplified flow:

- Accident date can be captured from cold replies.
- Fault.
- Medical treatment.
- Call timing.

The bot should not ask for a field that was already answered.

Warm Follow-Up / Re-Engagement

Used when someone started replying but stops mid-flow.

- Warm follow-up is faster than cold outreach.
- If still no response, the bot enters a question-specific re-engagement sequence.
- Progress is not reset.

Booking

The bot books through the GHL calendar when it has a safe call time.

High-risk booking/reschedule/confirmation actions go through the LLM decision gate where enabled.

Appointment types:

- `initial`: bot-created first specialist call.
- `qualified_follow_up`: human-created follow-up after intake looks qualified.
- `contract_review`: human-created agreement/contract review call.

Appointment Reminders

Reminder jobs are built from `preferredCallTimeIso`.

- Future-day appointments: morning, 1-hour, 5-minute reminders when timing allows.
- Same-day appointments: 1-hour and 5-minute reminders when timing allows.
- Very near appointments may only receive the 5-minute reminder.
- Appointment sync/update cancels stale reminders and rebuilds current reminders.

No-Show Recovery

Trigger:

- GHL no-show webhook or dashboard/admin mark-no-show.

Behavior:

- Sets `engagementStatus=missed_call`.
- Cancels normal appointment reminders.
- Uses backup time flow if backup exists and is still upcoming.
- Otherwise starts no-show recovery follow-up.
- Rebooked no-shows are tagged/stored as no-show rebooks.
- Third no-show escalates to human.

Human Escalation

Escalation pauses the bot and alerts Slack.

The bot can notify on new inbound replies while still paused so humans see additional PC responses.

Some soft escalations can auto-return after timeout if safe, but hard human work states remain paused.

Human Manual Text / Human Call Activity

These require GHL webhooks.

- Human outbound SMS tells the bot a human took over.
- Human call activity pauses the bot.
- Short/unknown-duration calls require an outcome tag.
- Missing call outcome creates a dashboard/Slack alert.

Call Outcome Tags

Team-facing tags/actions:

- `call_no_answer`

- `call_drop`
- `call_connected_follow_up`
- `return_to_bot`
- `NQ`
- `contract_sent`, `contract`, `contract_set`
- `signed`, `contract_signed`
- `follow_up`, `QR`, `human_hold`

Safety Stops

Hard stops:

- STOP / opt-out language.
- signed / contract_signed .
- NQ .
- GHL unsubscribed/DND/SMS blocked.
- Duplicate terminal same-phone contacts.

Manual pause/hold:

- follow_up
- QR
- human_hold
- call_connected_follow_up
- admin_pause

Contract pending:

- contract_sent , contract , contract_set stop intake/cold outreach but still allow appointment reminders and no-show recovery.

Slack Channels

Expected Slack channels:

- SMS escalation channel: general human escalation.
- Leads channel: urgent "call now" alerts.
- Booking channel: appointments, no-shows, rebooks.
- Bot errors channel: true system failures only.

Expected non-urgent items should go to dashboard, not Slack:

- DND/SMS blocked.
- Skipped jobs.
- Operational warnings where no human action is needed immediately.

Admin UI

Main dashboard is under:

- `/dashboard`

Utility pages:

- `/tester`
- `/backfill`
- `/integrations`
- `/training-status`
- `/review`
- `/dashboard-legacy`

Important dashboard concepts:

- Command center.
- Conversations/contact drilldown.
- Issues/stuck states.
- Appointments.
- Performance.
- Templates/A-B testing.
- Pause audit.
- Lifecycle map.

Owner Export Checklist

Source file: docs/handoff/OWNER_EXPORT_CHECKLIST.md

Owner Export Checklist

Use this checklist before handing the system to another operator or developer.

1. Decide What You Are Giving Them

Choose one:

- **Code only:** they get the bot logic and build their own database.
- **Code plus empty database:** best for a new operator.
- **Code plus cloned database:** only if they are legally allowed to receive the historical contacts/messages.
- **Code plus GHL workflow instructions:** usually required.
- **Full managed setup:** you deploy and operate it for them.

2. Protect Private Data

The database can contain:

- Names.
- Phone numbers.
- Emails.
- SMS conversations.
- Injury details.
- Legal/insurance context.
- Appointment history.
- Internal notes and decision logs.

Do not share a raw database unless you have permission and compliance approval.

If the recipient does not need historical data, give them an empty database.

3. Things To Export

Give them:

- Repo access or a private fork.
- This `docs/handoff/` folder.
- A sanitized env template, not your real `.env`.
- GoHighLevel workflow setup instructions.
- Slack channel setup instructions.
- Appointment title/tag guide from `docs/call-disposition-guide.md`.
- A database dump only if approved.
- A separate secrets package through a password manager.

Do not give them:

- Your live `.env`.
- Your live database URL in chat.
- Your OpenAI key in a document.
- Your Slack bot token in a document.
- Your GHL API token in a document.
- Your Render admin password in a document.

4. Secrets They Need To Provide Or Receive Securely

- DATABASE_URL
- ADMIN_PASSWORD
- WEBHOOK_SECRET
- GHL_API_TOKEN
- GHL_LOCATION_ID
- GHL_CALENDAR_ID
- SLACK_BOT_TOKEN
- SLACK_ESCALATION_CHANNEL
- SLACK_LEADS_CHANNEL
- SLACK_BOT_ERRORS_CHANNEL
- SLACK_BOOKING_CHANNEL
- OPENAI_API_KEY

5. GoHighLevel Items To Collect

From their GHL:

- Location/sub-account ID.
- Calendar ID.
- Private integration token or OAuth app installation.
- List of tags they use for states/firms.
- Timezone/state tag naming convention.
- Existing NR/no-response workflow location.
- Appointment workflow access.
- Ability to create webhooks in workflows.
- Ability to install Marketplace app/webhook for outbound message events if they need human manual text detection.

6. Slack Items To Collect

From their Slack:

- Workspace.
- Slack app bot token.
- Channel IDs for escalation, leads, booking, bot errors.
- Confirmation that bot was added to private channels.

7. Deployment Items To Collect

From Render/Railway/VPS:

- Public HTTPS app URL.
- Postgres connection string.
- Logs access.
- Environment variable access.
- Auto-deploy preference.
- Uptime monitor.

8. Handoff Safety Before They Go Live

Ask them to prove:

- `/health` is green.
- `/integrations` shows GHL, Slack, OpenAI, and calendar checks passing.
- GHL ping webhook lands in `/webhooks/ghl/ping-status`.
- Inbound SMS test is received.
- NR enrollment test sends exactly one first message.
- Appointment sync creates reminders.
- No-show webhook creates no-show jobs.
- STOP cancels everything.
- NQ/signed/follow-up/human_hold tags stop or pause as expected.

Recipient Setup Checklist

Source file: docs/handoff/RECIPIENT_SETUP_CHECKLIST.md

Recipient Setup Checklist

This is the step-by-step checklist for the receiving team.

Phase 1: Get The Code

1. Get access to the repo or a private fork.
2. Clone it:

```
git clone <repo-url>
cd asdleads-sms-bot
npm ci
npm test
npm run build
```

1. Confirm tests pass before changing anything.

Phase 2: Create Hosting

Recommended:

- Render Web Service.
- Render managed Postgres.

Required settings:

- Runtime: Node 20+
- Build command: `npm ci --include=dev && npm run build`
- Start command: `npm start`
- Health check path: `/health`
- Auto deploy: on for production branch if you want automatic updates.

Do not deploy as serverless. The bot has scheduled jobs and needs an always-on process.

Phase 3: Create Database

1. Create managed Postgres.
2. Add `DATABASE_URL` to the app service.
3. If importing an existing database snapshot, restore it before turning live SMS on.
4. If starting fresh, the app creates tables automatically at startup.

Phase 4: Add Environment Variables

Use `HANDOFF_ENV_TEMPLATE.env` as the checklist.

Keep `DRY_RUN=true` first.

Only set `DRY_RUN=false` after the validation plan passes.

Phase 5: Slack Setup

1. Create Slack app.
 2. Add bot token scopes needed to post messages, usually `chat:write`.
 3. Install app to workspace.
 4. Add bot to private channels.
 5. Copy channel IDs into env vars:
 - `SLACK_ESCALATION_CHANNEL`
 - `SLACK_LEADS_CHANNEL`
 - `SLACK_BOT_ERRORS_CHANNEL`
 - `SLACK_BOOKING_CHANNEL`
1. Test from `/integrations`.

Phase 6: GoHighLevel Setup

1. Create or obtain private integration token.
2. Add `GHL_API_TOKEN` , `GHL_LOCATION_ID` , and `GHL_CALENDAR_ID` .
3. Create all workflows in `GHL_WORKFLOWS_AND_WEBHOOKS.md` .
4. Include `webhookSecret` or `x-webhook-secret` with every webhook.
5. Test each webhook one by one.

Phase 7: OpenAI Setup

1. Create OpenAI API key.
2. Add `OPENAI_API_KEY`.
3. Use `gpt-5-mini` unless you intentionally choose another model.
4. Keep LLM fallback and decision gate enabled for risky actions:

```
LLM_FALLBACK_ENABLED=true  
LLM_DECISION_GATE_ENABLED=true  
LLM_DECISION_GATE_MODEL=gpt-5-mini
```

Phase 8: Local/Hosted Validation

1. Visit `/health` .
2. Visit `/integrations` .
3. Visit `/tester` .
4. Visit `/dashboard` .
5. Run the validation plan in `VALIDATION_TEST_PLAN.md` .

Phase 9: Go Live

1. Confirm only one bot service is connected live to the same GHL account.
2. Set `DRY_RUN=false`.
3. Trigger a real test contact only.
4. Watch Slack and dashboard.
5. Start with a small batch before any bulk NR activation.

Environment Variables

Source file: docs/handoff/ENVIRONMENT_VARIABLES.md

Environment Variables

Never commit real values. Put production values in Render/Railway/VPS environment variables.

Required App Variables

Variable	Required	Example	Notes
NODE_ENV	yes	production	Production store requires DATABASE_URL .
PORT	platform	3000	Render supplies this automatically.
HOST	yes	0.0.0.0	Use 0.0.0.0 in hosted production.
PUBLIC_BASE_URL	yes	https://your-app.onrender.com	Public app URL used in links and webhook docs.
ADMIN_PASSWORD	yes	secure random string	Protects admin APIs/dashboard.
WEBHOOK_SECRET	yes	secure random string	Must match GHL webhook custom data/header.
DATABASE_URL	yes	Postgres URL	Use managed Postgres. Do not share publicly.
DRY_RUN	yes	true then false	Start true. Production live SMS requires false.

GoHighLevel Variables

Variable	Required	Notes
GHL_API_TOKEN	yes	Private integration token or API token with contact, conversation, calendar, notes access.
GHL_LOCATION_ID	yes	The GHL sub-account/location ID.
GHL_CALENDAR_ID	yes	Calendar where Specialist calls are booked.
GHL_API_BASE	yes	Usually https://services.leadconnectorhq.com .
GHL_APP_BASE_URL	optional	Usually https://app.gohighlevel.com .

Slack Variables

Variable	Required	Notes
SLACK_BOT_TOKEN	yes	Bot token beginning with <code>xoxb-</code> .
SLACK_ESCALATION_CHANNEL	yes	SMS/human escalation channel ID or name.
SLACK_LEADS_CHANNEL	yes	Urgent call-now leads channel ID.
SLACK_BOT_ERRORS_CHANNEL	yes	True system errors only.
SLACK_BOOKING_CHANNEL	yes	Booking/no-show/rebook notices.
SLACK_SEND_IN_DRY_RUN	optional	Usually <code>false</code> .

Use channel IDs for private channels. The Slack bot must be invited into each private channel.

OpenAI Variables

Variable	Required	Recommended
OPENAI_API_KEY	yes if LLM enabled	Use recipient's own key.
OPENAI_CLASSIFIER_MODEL	yes	gpt-5-mini
LLM_FALLBACK_ENABLED	yes	true
LLM_MIN_CONFIDENCE	optional	0.85
LLM_CLARIFY_CONFIDENCE	optional	0.60
LLM_DECISION_GATE_ENABLED	yes	true
LLM_DECISION_GATE_MODEL	yes	gpt-5-mini
LLM_DECISION_GATE_MIN_CONFIDENCE	optional	0.82

Texting Window Variables

Variable	Required	Example
DEFAULT_TIMEZONE	yes	America/Chicago
DEFAULT_TEXTING_START	yes	08:00
DEFAULT_TEXTING_END	yes	21:00
STATE_TEXTING_WINDOWS_JSON	optional	{}
BOT_NAME	yes	William

STATE_TEXTING_WINDOWS_JSON can be used for stricter state rules.

Example:

```
{"FL":{"start":"08:00","end":"20:00"},"OK":{"start":"08:00","end":"20:00"}}
```

Training/Batch Variables

Only needed if importing historical SMS for training/audit:

Variable	Required	Notes
TRAINING_DB_PATH	optional	Defaults to <code>data/training.sqlite</code> .
AUTO_APPLY_LLM_BATCH	optional	Usually <code>false</code> .
BATCH_POLL_INTERVAL_MS	optional	Batch polling interval.

Secret Rotation Rule

When handing this system to another party:

1. Create new keys for them.
2. Do not give your existing keys.
3. If you accidentally shared a key in chat/email, rotate it immediately.

GoHighLevel Workflows And Webhooks

Source file: docs/handoff/GHL_WORKFLOWS_AND_WEBHOOKS.md

GoHighLevel Workflows And Webhooks

Every webhook should be a `POST` request to the recipient's deployed bot URL.

Base URL example:

```
https://your-app.onrender.com
```

If GHL lets you add headers, add:

```
x-webhook-secret: <WEBHOOK_SECRET>  
Content-Type: application/json
```

If GHL does not let you add headers, add this custom data field:

```
webhookSecret = <WEBHOOK_SECRET>
```

The bot accepts either.

Required Workflow 1: Test Ping

Purpose:

- Confirms GHL can reach the bot and the secret works.

Webhook URL:

```
POST /webhooks/ghl/ping
```

Custom data:

```
webhookSecret = <WEBHOOK_SECRET>  
contactId = {{contact.id}}  
name = {{contact.name}}  
phone = {{contact.phone}}
```

Check result:

```
GET /webhooks/ghl/ping-status
```

Required Workflow 2: NR / No Response Enrollment

Purpose:

- Starts the cold outreach flow after the team calls twice and marks the lead no response.

Preferred trigger:

- Opportunity moved to No Response stage, or
- Contact tag added `NR`, or
- Custom disposition `no response` / `NR`.

Webhook URL:

```
POST /webhooks/ghl/disposition
```

Aliases also accepted:

```
POST /webhooks/ghl/tag  
POST /webhooks/ghl/nr-tag
```

Custom data:

```
webhookSecret = <WEBHOOK_SECRET>  
contactId = {{contact.id}}  
name = {{contact.name}}  
phone = {{contact.phone}}  
email = {{contact.email}}  
disposition = NR  
tags = {{contact.tags}}  
leadSource = {{contact.source}}  
timezone = {{contact.timezone}}  
state = {{contact.state}}
```

Important:

- This should only fire after the team has actually made the required calls.
- Do not fire this for signed, NQ, QR, follow_up, or human_hold contacts.

Required Workflow 3: Inbound SMS

Purpose:

- Lets the bot see the PC's replies.

Trigger:

- Customer replied / inbound SMS / conversation message received.

Webhook URL:

```
POST /webhooks/ghl/inbound-sms
```

Custom data:

```
webhookSecret = <WEBHOOK_SECRET>
contactId = {{contact.id}}
name = {{contact.name}}
phone = {{contact.phone}}
email = {{contact.email}}
message = {{message.body}}
tags = {{contact.tags}}
leadSource = {{contact.source}}
timezone = {{contact.timezone}}
state = {{contact.state}}
```

If GHL uses different merge fields, map the latest inbound body into any of these accepted keys:

- message
- body
- text
- messageBody
- message_body

Required Workflow 4: Appointment Created / Updated

Purpose:

- Lets the bot know when the team manually creates or edits an appointment.
- Rebuilds reminders.
- Sends booking Slack notice.

Trigger:

- Appointment created.
- Appointment updated.
- Calendar appointment created/updated.

Webhook URL:

```
POST /webhooks/ghl/appointment-sync
```

Aliases also accepted:

```
POST /webhooks/ghl/appointment
POST /webhooks/ghl/appointment-created
POST /webhooks/ghl/appointment-updated
POST /webhooks/ghl/calendar-appointment
```

Custom data:

```
webhookSecret = <WEBHOOK_SECRET>
contactId = {{contact.id}}
appointmentId = {{appointment.id}}
startTime = {{appointment.start_time}}
status = {{appointment.status}}
title = {{appointment.title}}
tags = {{contact.tags}}
timezone = {{contact.timezone}}
```

Accepted appointment start fields:

- startTime
- start_time
- startsAt
- starts_at
- scheduledTime
- appointmentTime
- appointmentStartTime
- startDate
- appointment.startTime
- event.startTime

Appointment title conventions:

- ASD Initial Specialist Call - [Name]
- ASD Qualified Follow-Up Call - [Name]
- ASD Contract Review Call - [Name]

Required Workflow 5: Appointment No-Show

Purpose:

- Starts no-show recovery when the team marks an appointment no-show.

Trigger:

- Appointment status changed to no-show / missed / did not show.

Recommended webhook URL:

```
POST /webhooks/asd/no-show
```

Aliases also accepted:

```
POST /webhooks/ghl/no-show  
POST /webhooks/ghl/appointment-no-show  
POST /webhooks/asd/appointment-no-show
```

Custom data:

```
webhookSecret = <WEBHOOK_SECRET>  
contactId = {{contact.id}}  
appointmentId = {{appointment.id}}  
startTime = {{appointment.start_time}}  
status = no_show  
title = {{appointment.title}}  
tags = {{contact.tags}}  
timezone = {{contact.timezone}}
```

Check no-show diagnostics:

```
GET /api/admin/no-show-webhooks
```

Use the admin password header if required:

```
x-admin-password: <ADMIN_PASSWORD>
```

Required Workflow 6: Bot Control Tags

Purpose:

- Lets human team tags control bot behavior.

Trigger:

- Contact tag added.

Webhook URL:

```
POST /webhooks/ghl/bot-control
```

Simple setup:

- Make one workflow per important tag.
- Hardcode the `action` value for that tag.

Custom data:

```
webhookSecret = <WEBHOOK_SECRET>
contactId = {{contact.id}}
name = {{contact.name}}
phone = {{contact.phone}}
tags = {{contact.tags}}
action = <TAG_OR_ACTION_NAME>
actor = {{user.name}}
```

Supported action/tag values:

- `call_no_answer`
- `call_drop`
- `call_connected_follow_up`
- `return_to_bot`
- `NQ`
- `contract_sent`
- `contract`
- `contract_set`
- `signed`
- `contract_signed`
- `follow_up`
- `QR`
- `human_hold`
- `pause_bot`
- `urgent_call_now`
- `mark_no_show`
- `ensure_appointment_reminders`
- `refresh_timezone`

Use `NQ`, not `not_qualified`.

Required Workflow 7: Human Call Activity

Purpose:

- Pauses the bot when a human places a call.
- Requires the team to record an outcome if the call is short or unclear.

Trigger:

- Outbound call activity.
- Manual call started/completed.

Webhook URL:

```
POST /webhooks/ghl/human-active
```

Custom data:

```
webhookSecret = <WEBHOOK_SECRET>
contactId = {{contact.id}}
action = manual_call
callStatus = {{call.status}}
callDuration = {{call.duration}}
callDirection = outbound
actor = {{user.name}}
```

If duration is missing or under 60 seconds, the bot treats it as needing a call outcome.

Required Workflow 8: Human Manual Text / Outbound Message

Purpose:

- Lets the bot know a human manually texted the PC.
- Without this, the bot cannot reliably know what the human said.

Preferred method:

- GHL Marketplace app webhook event for outbound conversation messages.

Webhook URL:

```
POST /webhooks/asd/human-outbound
```

Alias also accepted:

```
POST /webhooks/ghl/human-outbound
```

Required event:

```
OutboundMessage
```

Required app/webhook scope:

```
conversations/message.readonly
```

If using a normal GHL workflow instead of Marketplace webhook, custom data should include:

```
webhookSecret = <WEBHOOK_SECRET>
contactId = {{contact.id}}
type = OutboundMessage
message = {{message.body}}
messageType = SMS
actor = {{user.name}}
```

If GHL does not expose a normal "manual SMS sent" workflow trigger, use the Marketplace app event. If that is not installed, the fallback is for the team to tag `call_connected_follow_up`, `return_to_bot`, or another bot-control tag manually.

Optional Workflow 9: Missed Call

Purpose:

- Starts missed-call follow-up if a scheduled call was missed outside the no-show workflow.

Webhook URL:

```
POST /webhooks/ghl/missed-call
```

Custom data:

```
webhookSecret = <WEBHOOK_SECRET>
contactId = {{contact.id}}
appointmentId = {{appointment.id}}
callTime = {{appointment.start_time}}
status = missed
```

Testing Each Webhook

For every workflow:

1. Trigger it on a test contact only.
2. Check Render logs.
3. Check `/integrations`.
4. Check dashboard contact timeline.
5. Check Slack if it is supposed to alert.
6. Confirm no duplicate messages were sent.

Database Transfer

Source file: docs/handoff/DATABASE_TRANSFER.md

Database Transfer Guide

This bot uses Postgres in production.

The database contains private lead/contact/SMS data. Treat it as sensitive.

What Is In The Database

Tables:

- `contacts`
- `jobs`
- `messages`
- `escalations`
- `decision_logs`
- `webhook_events`
- `settings`

The `contacts` and `messages` tables can contain personal, medical, legal, and insurance information.

Option A: Start With Empty Database

Best for a new operator.

Steps:

1. Create new Postgres database.
2. Set `DATABASE_URL` .
3. Start app.
4. App creates tables automatically.
5. Configure GHL workflows.
6. Start live traffic after testing.

No historical contacts/messages are transferred.

Option B: Clone Full Database

Use only if the recipient is allowed to receive all historical data.

Export

Run from a trusted machine with Postgres tools installed:

```
mkdir -p private-handoff
pg_dump "$DATABASE_URL" \
  --format=custom \
  --no-owner \
  --no-acl \
  --file "private-handoff/asdleads_sms_bot_$(date +%Y-%m-%d).dump"
```

Encrypt before sending:

```
gpg -c --cipher-algo AES256 "private-handoff/asdleads_sms_bot_$(date +%Y-%m-%d).dump"
```

Send the `.gpg` file separately from the password.

Do not commit database dumps.

Restore

On the recipient environment:

```
gpg -o private-handoff/asdleads_sms_bot.dump -d private-handoff/asdleads_sms_bot_YYYY-MM-DD.dump.gpg
pg_restore \
  --clean \
  --if-exists \
  --no-owner \
  --dbname "$NEW_DATABASE_URL" \
  private-handoff/asdleads_sms_bot.dump
```

Then start the app and check:

```
GET /health
GET /dashboard
```

Option C: Clone Only Bot Logic, Not Contacts

Recommended if the second operator has their own GHL leads.

Steps:

1. Do not restore old contacts/messages/jobs.
2. Deploy code.
3. Configure new GHL location and calendar.
4. Start with fresh NR enrollment.

Important Cutover Warning

Never have two live services sending SMS for the same GHL location/contact list.

Safe migration:

1. Old bot stays live.
2. New bot is deployed with `DRY_RUN=true`.
3. New bot webhooks tested on test contact only.
4. Old bot webhooks disabled.
5. New bot `DRY_RUN=false`.
6. GHL workflows switched to new URL.

Jobs Table Warning

The `jobs` table contains pending scheduled messages.

If cloning a database for another operator, decide whether pending jobs should come over:

- If they are cloning the same live operation, keep jobs.
- If they are starting a new operation, clear pending jobs before live use.

Clear jobs only if you are sure:

```
update jobs
set status = 'cancelled',
    data = data || '{"cancelReason":"handoff reset"}'::jsonb,
    updated_at = now()
where status = 'pending';
```

Training Database

Historical training import may also exist locally:

```
data/training.sqlite
```

This is not the production bot memory. Transfer it only if they need the review/training examples.

It may also contain private SMS text, so treat it as sensitive.

Validation Test Plan

Source file: docs/handoff/VALIDATION_TEST_PLAN.md

Validation Test Plan

Run this before switching `DRY_RUN=false` .

1. Local Code Checks

```
npm ci  
npm test  
npm run build
```

Expected:

- All tests pass.
- Dashboard build succeeds.

2. Health Check

Open:

```
https://YOUR-APP/health
```

Expected:

- `ok: true`
- storage type is `postgres`
- no database errors

3. Integration Page

Open:

<https://YOUR-APP/integrations>

Expected:

- GHL read access works.
- Slack token valid.
- Calendar configured.
- OpenAI configured.
- Dry run status is clear.

4. GHL Ping Test

Trigger `/webhooks/ghl/ping` .

Then open:

```
https://YOUR-APP/webhooks/ghl/ping-status
```

Expected:

- `authorized: true`
- contact id present
- payload keys visible

5. NR Enrollment Test

Use a test GHL contact.

Trigger NR/no-response workflow.

Expected:

- Contact appears in dashboard.
- One initial SMS is sent or queued.
- No duplicate initial SMS.
- Cold outreach jobs exist.

6. Inbound Reply Test

Reply as the PC:

May 10th

Expected:

- Accident date saved.
- Bot asks next needed qualification question.

Reply:

The other driver

Expected:

- Fault saved as not at fault.
- Bot asks medical question.

Reply:

Yes I went to the hospital

Expected:

- Medical saved.
- Bot asks for call timing.

7. Call Now Test

Reply:

Call me now

Expected:

- Urgent Slack alert goes to leads channel.
- Bot does not keep asking qualification questions.
- Human team calls.

8. Exact Booking Test

Reply:

Tomorrow at 3pm

Expected:

- GHL appointment created.
- Booking Slack notice sent.
- Reminder jobs scheduled.
- Timezone looks correct.

9. Manual Appointment Sync Test

Human creates appointment in GHL.

Expected:

- `/webhooks/ghl/appointment-sync` processes it.
- Contact becomes `call_scheduled`.
- Reminders are scheduled.
- Booking Slack notice sent once.

10. No-Show Test

Mark test appointment no-show in GHL.

Expected:

- `/webhooks/asd/no-show` receives the event.
- Contact becomes `missed_call`.
- Old reminders are cancelled.
- No-show follow-up jobs are scheduled.
- Dashboard no-show log shows `jobs_scheduled` or `backup_flow_started`.

11. Human Manual Text Test

Human sends an SMS manually from GHL.

Expected:

- `/webhooks/asd/human-outbound` receives the event.
- Message is stored as `human_outbound`.
- Bot pauses and schedules human reply timeout.
- If the human text contains a call time in a scheduling state, bot can assist booking.

12. Human Call Activity Test

Human places short outbound call.

Expected:

- `/webhooks/ghl/human-active` receives it.
- Bot pauses.
- Call outcome watchdog is scheduled.
- If no outcome tag is added, dashboard/Slack shows no call disposition recorded.

13. Stop/NQ/Signed Tests

STOP:

- Reply `STOP` .
- Expected: opted out, jobs cancelled, no Slack error for expected GHL unsubscribe block.

NQ:

- Add `NQ` .
- Expected: bot stops, no more follow-up.

Signed:

- Add `signed` .
- Expected: terminal stop, reminders/no-show recovery cancelled.

14. Lead-Requested Pause Test

Reply:

Not a good time. I'll text when I'm free. Please don't blow up my phone

Expected:

- One polite acknowledgement.
- All pending jobs cancelled.
- Team gets an escalation/alert.
- Bot does not opt them out.
- Bot resumes only when lead texts again.

15. Duplicate Safety Test

Create or simulate duplicate same-phone contacts where one is NQ/signed.

Expected:

- Bot does not continue texting the wrong duplicate.
- Dashboard shows duplicate issue if needed.

Go-Live Gate

Only set `DRY_RUN=false` after all required tests pass.

After go-live:

- Watch Slack for 1 hour.
- Watch dashboard issues.
- Send only a small first batch.
- Do not bulk enroll hundreds of contacts until duplicate and DND behavior looks clean.

Repo Update Strategy

Source file: docs/handoff/REPO_UPDATE_STRATEGY.md

Repo Update Strategy

This explains how another operator can receive future updates while protecting production.

Option 1: Same Private Repo, Separate Environment

How it works:

- Add recipient/dev as GitHub collaborator.
- They deploy the same repo to their own Render service.
- Their environment variables point to their own GHL, Slack, OpenAI, and database.
- Auto-deploy from `main` can update them automatically.

Pros:

- Easiest to keep updated.
- One source of truth.
- Every push can deploy everywhere if configured.

Cons:

- They see the full repo history.
- A bad push could affect all connected auto-deploy environments.
- Harder to customize per recipient.

Use this only with trusted collaborators and good test discipline.

Option 2: Private Fork With Upstream

How it works:

- Recipient forks the repo privately.
- Their deployment points to their fork.
- Original repo is added as `upstream`.
- They pull updates when ready.

Commands:

```
git remote add upstream <original-repo-url>
git fetch upstream
git merge upstream/main
npm test
npm run build
git push origin main
```

Pros:

- Safer for production.
- Recipient controls when updates deploy.
- Recipient can customize without changing your production.

Cons:

- Someone must pull/merge updates.
- Merge conflicts are possible.

Recommended for most outside operators.

Option 3: Release Tags

How it works:

- You create stable release tags:

```
git tag handoff-2026-05-14  
git push origin handoff-2026-05-14
```

- Recipient deploys a specific tag or branch.

Pros:

- Stable.
- Easy rollback.
- Good for multiple clients/operators.

Cons:

- Requires release discipline.

Recommended if this becomes a repeatable product.

Option 4: Template Repo

How it works:

- Create a clean template repo without client-specific docs/history.
- New operators create a repo from the template.

Pros:

- Cleanest handoff.
- No accidental client-specific files.

Cons:

- More work to maintain.
- Updates need a release/merge strategy.

Strong Recommendation

For one trusted operator:

- Use a private fork.
- Keep your repo as upstream.
- Pull updates manually after tests pass.

For many operators:

- Create a clean product/template repo.
- Use release tags.
- Keep each operator on their own database and env vars.

Auto-Deploy Warning

Auto-deploy is powerful but risky.

If a recipient points Render directly to `main`, every push to `main` can deploy to their production.

Safer:

- Deploy from a `production` branch.
- Merge into `production` only after tests.
- Keep `main` as active development.

Example:

```
git checkout -b production
git push origin production
```

Then set Render branch to `production`.

Rollback

If a deploy breaks:

1. Go to Render deploy history.
2. Roll back to previous deploy.
3. Or revert the Git commit:

```
git revert <bad-commit-sha>
git push
```

1. Verify `/health`, `/dashboard`, and a test webhook.

Export Package Checklist

Source file: docs/handoff/EXPORT_PACKAGE_CHECKLIST.md

Export Package Checklist

Use this when preparing a complete handoff.

Files/Folders To Include

Include:

- Full repo or private fork.
- docs/handoff/
- docs/handoff/Accident_Support_Desk_Complete_Handoff_Guide.pdf
- docs/handoff/COMPLETE_HANDOFF_GUIDE.md
- docs/handoff/HANDOFF_ENV_TEMPLATE.env
- README.md
- RENDER_DEPLOY_GUIDE.md
- GO_LIVE_GUIDE.md
- docs/PRODUCTION_RUNBOOK.md
- docs/call-disposition-guide.md
- docs/team-guide.md
- docs/bot-logic-audit.md
- docs/lead-lifecycle-map.md
- render.yaml
- .env.example

Do not include:

- .env
- data/
- reports/ if reports contain private info
- database dumps unless encrypted and approved
- screenshots with private keys/tokens

Suggested Private Handoff Folder

Create this outside the repo:

```
private-handoff/  
  database/  
  secrets/  
  screenshots/  
  ghl-workflow-notes/
```

Keep `private-handoff/` out of Git.

Suggested Public/Safe Handoff Folder

This repo folder is safe to commit:

docs/handoff/

It should contain instructions and templates only.

What To Tell The Recipient

Tell them:

1. Start with `docs/handoff/Accident_Support_Desk_Complete_Handoff_Guide.pdf`.
2. Use `RECIPIENT_SETUP_CHECKLIST.md`.
3. Use `HANDOFF_ENV_TEMPLATE.env` for variables.
4. Configure GHL workflows exactly from `GHL_WORKFLOWS_AND_WEBHOOKS.md`.
5. Do not go live until `VALIDATION_TEST_PLAN.md` passes.

Sanitized Environment Template

Copy this into the recipient hosting service and fill in their own values. Never reuse Collins' live secrets.

```
NODE_ENV=production
HOST=0.0.0.0
PUBLIC_BASE_URL=https://YOUR-APP-DOMAIN.example
ADMIN_PASSWORD=REPLACE_WITH_SECURE_PASSWORD
WEBHOOK_SECRET=REPLACE_WITH_SECURE_RANDOM_SECRET
DATABASE_URL=REPLACE_WITH_POSTGRES_CONNECTION_STRING
DRY_RUN=true

GHL_API_TOKEN=REPLACE_WITH_GHL_PRIVATE_INTEGRATION_TOKEN
GHL_LOCATION_ID=REPLACE_WITH_GHL_LOCATION_ID
GHL_CALEDAR_ID=REPLACE_WITH_GHL_CALEDAR_ID
GHL_API_BASE=https://services.leadconnectorhq.com
GHL_APP_BASE_URL=https://app.gohighlevel.com

SLACK_BOT_TOKEN=REPLACE_WITH_SLACK_BOT_TOKEN
SLACK_ESCALATION_CHANNEL=REPLACE_WITH_CHANNEL_ID_OR_NAME
SLACK_LEADS_CHANNEL=REPLACE_WITH_CHANNEL_ID_OR_NAME
SLACK_BOT_ERRORS_CHANNEL=REPLACE_WITH_CHANNEL_ID_OR_NAME
SLACK_BOOKING_CHANNEL=REPLACE_WITH_CHANNEL_ID_OR_NAME
SLACK_SEND_IN_DRY_RUN=false

OPENAI_API_KEY=REPLACE_WITH_OPENAI_API_KEY
OPENAI_CLASSIFIER_MODEL=gpt-5-mini
LLM_FALLBACK_ENABLED=true
LLM_MIN_CONFIDENCE=0.85
LLM_CLARIFY_CONFIDENCE=0.60
LLM_DECISION_GATE_ENABLED=true
LLM_DECISION_GATE_MODEL=gpt-5-mini
LLM_DECISION_GATE_MIN_CONFIDENCE=0.82

DEFAULT_TIMEZONE=America/Chicago
DEFAULT_TEXTING_START=08:00
DEFAULT_TEXTING_END=21:00
STATE_TEXTING_WINDOWS_JSON={}
BOT_NAME=William
```

Final Handoff Checklist

- Repo access granted or private fork created.
- Recipient hosting and Postgres created.
- Recipient env vars entered with their own secrets.
- GHL workflows created and tested.
- Slack channels/app installed and tested.
- OpenAI API key added if LLM gate is enabled.
- Database restored only if needed and legally approved.
- DRY_RUN=true for all tests.
- Validation plan passed.
- DRY_RUN=false only after old automation conflicts are disabled.
- Uptime monitoring enabled on /health.